

Spring Framework

& Spring Boot — Complete Questions + All Annotations

Spring Core • Spring Boot • Spring MVC • Spring Data JPA • Spring Security • Spring AOP • Spring Transactions • All Annotations Reference

#	Section	#	Section
S1	Spring Core — IoC, DI, Beans, Lifecycle	S9	Spring AOP — Aspects, Advice, Pointcuts
S2	Spring Boot — Core, Auto-config, Properties	S10	Spring Testing — JUnit, Mockito, MockMvc
S3	Spring Boot — Starters, Actuator, DevTools	S11	Spring Transactions — Deep Dive
S4	Spring MVC — DispatcherServlet, Controllers	S12	Spring REST — Advanced Topics
S5	Spring MVC — REST, Validation, CORS	S13	ALL Spring Core Annotations
S6	Spring Data JPA — Entities, Repositories	S14	ALL Spring Boot Annotations
S7	Spring Data JPA — Queries, Pagination	S15	ALL Spring MVC / REST Annotations
S8	Spring Security — JWT, Auth, Filters	S16	ALL JPA / Security / AOP Annotations

Questions only — no answers. Annotations section includes purpose, package, and usage context.

IoC Container & ApplicationContext

1. What is the Spring Framework? What are its core modules?
2. What is Inversion of Control (IoC)? How does Spring implement it?
3. What is the difference between BeanFactory and ApplicationContext?
4. What are the implementations of ApplicationContext — name 5?
5. What is the Spring IoC container startup sequence?
6. What is a Spring Bean? What makes a class a Spring Bean?
7. What is component scanning? How does Spring discover beans automatically?
8. What is the difference between XML-based, annotation-based, and Java-based Spring configuration?
9. What is a @Configuration class? How is it different from a regular class annotated with @Component?
10. Can you have multiple @Configuration classes in one application?
11. What is ApplicationContextAware? When would you use it?
12. What is EnvironmentAware and how is it used?
13. What is a parent-child ApplicationContext? Where is it used in Spring MVC?
14. What is @Import? How do you use it to import other configuration classes?
15. What is @ImportResource? When would you use it?
16. What is @PropertySource? How do you load a custom .properties file?
17. What is @ComponentScan? What are its attributes (basePackages, excludeFilters)?

Dependency Injection

1. What is Dependency Injection? What are its benefits over manual object creation?
2. What are the 3 types of Dependency Injection in Spring?
3. What is Constructor Injection? Why does the Spring team recommend it?
4. What is Field Injection? Why is it not recommended for production code?
5. What is Setter Injection? When would you prefer it?
6. What is @Autowired? How does Spring resolve it by type?
7. What happens when multiple beans of the same type exist and you use @Autowired?
8. What is @Qualifier? How is it used alongside @Autowired?
9. What is @Primary? How does it differ from @Qualifier?
10. What is a circular dependency? Give a real example. How does Spring handle it?
11. What is @Lazy? How does it help with circular dependency?
12. What is @Bean? How is it different from @Component?
13. What is the difference between @Component, @Service, @Repository, and @Controller?
14. What is @Value? How do you inject a property from application.properties?
15. What is @ConfigurationProperties? How is it different from @Value?
16. What is SpEL (Spring Expression Language)? Give an example using @Value with SpEL.
17. What is a stereotype annotation in Spring?

18. What is method injection? When would you use it?
19. What is lookup method injection? What problem does it solve for Prototype in Singleton?
20. What is @Autowired on a List or Map? What does Spring inject?

Bean Lifecycle & Scopes

1. What is the complete Spring Bean lifecycle — all phases from instantiation to destruction?
2. What is @PostConstruct? At which lifecycle phase does it run?
3. What is @PreDestroy? At which lifecycle phase does it run?
4. What is InitializingBean? What method does it require?
5. What is DisposableBean? What method does it require?
6. What is the difference between @PostConstruct and InitializingBean? Which runs first?
7. What is BeanPostProcessor? What are its two methods and when do they run?
8. What is BeanFactoryPostProcessor? When does it run compared to BeanPostProcessor?
9. What are the Spring Bean scopes? List all 6.
10. What is Singleton scope? Is a Singleton bean thread-safe? Why or why not?
11. What is Prototype scope? Who is responsible for destroying a Prototype bean?
12. What are the web-aware scopes: Request, Session, Application, WebSocket?
13. What happens when a Singleton bean injects a Prototype-scoped bean?
14. How do you inject a fresh Prototype bean into a Singleton — 3 different approaches?
15. What is a scoped proxy? How is it created with @Scope(proxyMode=)?
16. What is @DependsOn? When would you use it?
17. What is FactoryBean? How is it different from a regular bean?
18. What is SmartInitializingSingleton? When does it run?
19. What is @Conditional? What are the different conditional annotations?

Spring Boot Fundamentals

1. What is Spring Boot? What problems does it solve over plain Spring Framework?
2. What is convention over configuration? How does Spring Boot implement it?
3. What is `@SpringBootApplication`? What 3 annotations does it combine?
4. How does `SpringApplication.run()` work internally — step by step?
5. What is a `SpringApplicationEvent`? Name 6 application events in order.
6. What is the difference between Spring Boot 2.x and Spring Boot 3.x (Jakarta EE)?
7. What is an embedded server? How does Spring Boot embed Tomcat?
8. How do you switch the embedded server from Tomcat to Jetty or Undertow?
9. What is a fat JAR (uber JAR)? What is the `spring-boot-maven-plugin` repackaging goal?
10. What is `@EnableAutoConfiguration`? How does it work at runtime?
11. What is the `spring.factories` file (Boot 2) and `AutoConfiguration.imports` (Boot 3)?
12. What is `@ConditionalOnClass`? Give a real example of when it activates.
13. What is `@ConditionalOnMissingBean`? Why is it important in auto-configuration?
14. What is `@ConditionalOnProperty`? Give an example.
15. What is `@ConditionalOnWebApplication`? What is `@ConditionalOnNotWebApplication`?
16. What is `@ConditionalOnExpression`?
17. How do you exclude a specific auto-configuration class?
18. How do you debug which auto-configurations were applied (positive matches) and which were not?
19. What is `@AutoConfiguration` (Spring Boot 3)? How is it different from `@Configuration`?
20. How do you create a custom Spring Boot auto-configuration?

Application Properties & Profiles

1. What is `application.properties` vs `application.yml`? What are the pros and cons of each?
2. What is the order of property sources in Spring Boot (highest to lowest priority)?
3. What is `@Value`? What types can it inject? What is the `${}` vs `#{}` syntax?
4. What is `@ConfigurationProperties`? What is its advantage over multiple `@Value`?
5. What is `@EnableConfigurationProperties`?
6. What is relaxed binding in `@ConfigurationProperties`?
7. What is `@Validated` on `@ConfigurationProperties`? What does it enable?
8. How do you define a Spring Profile?
9. What is `application-{profile}.properties`? How does it work?
10. How do you activate a profile — 3 different ways?
11. What is `@Profile`? How do you create profile-specific beans?
12. What is `spring.profiles.active` vs `spring.profiles.include`?
13. What is a default profile?
14. What is `@ActiveProfiles` in tests?

15. How do you pass properties at runtime via command-line arguments?
16. How do you override a Spring Boot property with an environment variable?
17. What is `spring.config.location`? What is `spring.config.name`?
18. What is `@PropertySource`? Can it load YAML files?
19. What is a config tree (`spring.config.import`)?
20. What is Spring Cloud Config Server? How does a client bootstrap from it?

Spring Boot Starters

1. What is a Spring Boot Starter? What is its internal structure (pom.xml + auto-config)?
2. What does spring-boot-starter-web include? What is auto-configured?
3. What does spring-boot-starter-data-jpa include?
4. What does spring-boot-starter-security include?
5. What does spring-boot-starter-test include?
6. What does spring-boot-starter-actuator include?
7. What does spring-boot-starter-validation include?
8. What does spring-boot-starter-mail include?
9. What does spring-boot-starter-cache include?
10. What does spring-boot-starter-data-redis include?
11. What does spring-boot-starter-amqp include?
12. What does spring-boot-starter-webflux include?
13. What is spring-boot-starter-parent? What does it manage?
14. What is spring-boot-dependencies BOM? What is a BOM in Maven?
15. How do you create a custom Spring Boot Starter?
16. What is the difference between a Starter and an auto-configuration module?

Spring Boot Actuator

1. What is Spring Boot Actuator? Why is it needed in production?
2. What is the /actuator/health endpoint? What does it return by default?
3. What is HealthIndicator? How do you create a custom one?
4. What is the /actuator/info endpoint? How do you populate app info?
5. What is the /actuator/metrics endpoint? What metrics are available?
6. What is the /actuator/beans endpoint? What does it expose?
7. What is the /actuator/env endpoint? What does it show?
8. What is the /actuator/mappings endpoint?
9. What is the /actuator/loggers endpoint? Can you change log level at runtime?
10. What is the /actuator/conditions endpoint?
11. What is the /actuator/threaddump endpoint?
12. What is the /actuator/heapdump endpoint?
13. What is the /actuator/shutdown endpoint? How do you enable it?
14. What is management.endpoints.web.exposure.include=*?
15. How do you secure Actuator endpoints from public access?
16. What is Micrometer? What is its relationship with Actuator?
17. What is a Counter, Gauge, Timer, and DistributionSummary in Micrometer?
18. How do you integrate Spring Boot Actuator with Prometheus and Grafana?

19. What is the management.server.port property?
20. How do you change the actuator base path from /actuator?

Spring Boot DevTools & Testing Utilities

1. What is spring-boot-devtools? What features does it provide?
2. What is automatic restart in DevTools? How is it faster than a cold restart?
3. What is LiveReload in DevTools?
4. What is the difference between restart classloader and base classloader in DevTools?
5. What is remote DevTools?
6. What is CommandLineRunner? When does it run?
7. What is ApplicationRunner? How does it differ from CommandLineRunner?
8. What is @Order on CommandLineRunner? Why does it matter?
9. What is ApplicationArguments? What is the difference between option and non-option args?
10. What is a Spring Boot banner? How do you customise it?

Spring MVC Architecture

1. What is Spring MVC? What pattern does it follow?
2. What is the DispatcherServlet? What is its role?
3. Explain the complete Spring MVC request-processing lifecycle step by step.
4. What is HandlerMapping? What are the built-in implementations?
5. What is HandlerAdapter? What does it do?
6. What is ViewResolver? Name 3 implementations.
7. What is a View in Spring MVC?
8. What is ModelAndView? When would you use it?
9. What is the Model? What is @ModelAttribute?
10. What is @RequestMapping? What are all its attributes?
11. What is the difference between @Controller and @RestController?
12. What is @ResponseBody? What happens when you add it to a method?
13. What is HttpMessageConverter? How does Spring choose one?
14. What is Jackson? How does Spring MVC use it for JSON serialization?
15. What is content negotiation? How is it configured?
16. What is @RequestHeader? Give an example use case.
17. What is @CookieValue? How do you read a cookie in a controller?
18. What is @SessionAttribute and @SessionAttributes?
19. What is @MatrixVariable? When is it used?
20. What is Flash attribute? What is RedirectAttributes?
21. What is InternalResourceViewResolver?
22. What is HandlerInterceptor? What are preHandle, postHandle, afterCompletion?
23. What is WebMvcConfigurer? How is it used for global MVC customisation?
24. What is @EnableWebMvc? When do you need it in Spring Boot?

Request Mapping & Parameter Binding

1. What is @PathVariable? How do you use it with regex constraints?
2. What is @RequestParam? What are its required, defaultValue, and name attributes?
3. What is @RequestBody? How does Jackson deserialise the JSON?
4. What is @ResponseStatus? How does it affect the HTTP response?
5. What is ResponseEntity? When do you use it vs returning an object directly?
6. What is MultipartFile? How do you handle file upload in a Spring controller?
7. What is @RequestPart? How does it differ from @RequestParam for multipart?
8. What is @InitBinder? When would you use it?
9. What is WebDataBinder?
10. What is UriComponentsBuilder? How do you build a URI programmatically?

11. What is a forward vs redirect in Spring MVC? How do you do each?
12. What is the difference between return 'redirect:/path' and ResponseEntity with 302?

REST API Design in Spring

1. What is REST? What are its 6 architectural constraints?
2. What are the HTTP methods and their semantics in a RESTful API?
3. What is idempotency? Which HTTP methods are idempotent?
4. What is the correct HTTP status code for each operation: create, read, update, delete?
5. What is the difference between PUT and PATCH semantics?
6. What is the Location header? When should you return it?
7. What is HATEOAS? What is spring-hateoas?
8. What is API versioning? What are the different strategies (URI, header, param)?
9. What is `@CrossOrigin`? What attributes does it have?
10. What is CORS? Why do browsers enforce it?
11. How do you configure global CORS in Spring Boot?
12. What is `CorsConfigurationSource`?
13. What is `HttpStatus`? How many status code groups are there?
14. What is the `ProblemDetail` class (RFC 7807) introduced in Spring 6?
15. What is `@RestControllerAdvice`? How is it different from `@ControllerAdvice`?

Bean Validation & Exception Handling

1. What is Bean Validation (JSR-380)? What dependency enables it?
2. What is `@Valid` vs `@Validated`? What is the difference?
3. What is `MethodArgumentNotValidException`? How do you handle it globally?
4. What is `ConstraintViolationException`? When does it differ from `MethodArgumentNotValidException`?
5. What is `@ExceptionHandler`? How is it scoped to a single controller vs global?
6. What is `@ControllerAdvice`? What is `@RestControllerAdvice`?
7. What is the execution order when multiple `@ExceptionHandler` methods exist?
8. What is `HandlerExceptionResolver`? Name 3 built-in implementations.
9. What is `DefaultHandlerExceptionResolver`? What exceptions does it handle?
10. How do you return a structured error response with timestamp, status, message, path?
11. What is `@ResponseStatus` on an exception class? What does it do?
12. How do you create a custom constraint annotation (validator)?
13. What is `@Constraint`? What is `ConstraintValidator`?
14. What is groups in Bean Validation? When would you use validation groups?
15. What is `@Valid` on nested objects (cascaded validation)?

JPA Entity Mapping

1. What is JPA? What is Hibernate? How do they relate?
2. What is an ORM? What problem does it solve?
3. What is a JPA Entity? What rules must a class follow to be an entity?
4. What is @Entity? What is @Table? How do you specify the table name?
5. What is @Id? What is @GeneratedValue? What are the 4 generation strategies?
6. What is @Column? What attributes does it support (name, nullable, unique, length)?
7. What is @Transient? When do you use it?
8. What is @Enumerated? Explain STRING vs ORDINAL and why ORDINAL is dangerous.
9. What is @Lob? When would you use it?
10. What is @Version? What is optimistic locking?
11. What is pessimistic locking? What LockModeType values exist?
12. What is @Embedded and @Embeddable? Give an example.
13. What is @AttributeOverride?
14. What is a composite primary key? How do you use @IdClass vs @EmbeddedId?
15. What is @CreationTimestamp and @UpdateTimestamp (Hibernate annotations)?
16. What is @NaturalId in Hibernate?
17. What is @ColumnDefault in Hibernate?
18. What is the equals() and hashCode() contract for JPA entities? Why is it important?
19. Why should you not use auto-generated ID in equals/hashCode for JPA entities?

JPA Relationships & Fetch Strategies

1. What is @ManyToOne? What is its default fetch type?
2. What is @OneToMany? What is its default fetch type? What is mappedBy?
3. What is @OneToOne? What are the mapping strategies?
4. What is @ManyToMany? How do you customise the join table with @JoinTable?
5. What is the owning side and inverse side of a relationship?
6. What is CascadeType? Explain all values: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL.
7. What is orphanRemoval=true? How is it different from CascadeType.REMOVE?
8. What is FetchType.EAGER vs FetchType.LAZY?
9. What is the N+1 select problem? Give a concrete example with Code.
10. What are the 3 solutions to the N+1 problem?
11. What is JOIN FETCH in JPQL? Write an example query.
12. What is @EntityGraph? How do you define and use it?
13. What is @BatchSize (Hibernate)? How does it reduce N+1 queries?
14. What is a bidirectional relationship? How do you keep both sides in sync?
15. What is @JoinColumn? What FK column does it create?

16. What is @JoinTable? Write a @ManyToMany example with a custom join table.
17. What is @MapsId? When is it used for shared primary keys?
18. What is the difference between orphanRemoval and cascade delete at DB level?

JPA / Hibernate Internals

1. What are the 4 entity states in JPA: Transient, Managed, Detached, Removed?
2. What is the persistence context? What is its lifecycle in Spring?
3. What is the first-level cache (session cache) in Hibernate?
4. What is the second-level cache? What providers are supported?
5. What is dirty checking in Hibernate? When does it happen?
6. What is FlushMode? Explain AUTO, COMMIT, ALWAYS, MANUAL.
7. What is the difference between merge() and persist()?
8. What is detach() and refresh()?
9. What is LazyInitializationException? When does it occur? How do you fix it?
10. What is the Open Session in View anti-pattern?
11. What is hibernate.ddl-auto? Explain all 5 options.
12. What is the difference between ddl-auto=update and validate?
13. What is Flyway? What is Liquibase? Why are they better than ddl-auto in production?
14. What is the Hibernate second-level cache? How do you enable @Cacheable for an entity?

Spring Data JPA Repositories

1. What is Spring Data JPA? What does it eliminate?
2. What is the difference between `JpaRepository`, `CrudRepository`, and `PagingAndSortingRepository`?
3. What is `ListCrudRepository` and `ListPagingAndSortingRepository` (Spring Data 3)?
4. What methods does `JpaRepository` provide out of the box? Name 15.
5. What is `save()`? How does Spring Data decide whether to INSERT or UPDATE?
6. What is `saveAll()`? What is `saveAndFlush()`? What is `saveAllAndFlush()`?
7. What is `findById()` vs `getReferenceById()`? What is the difference?
8. What is `existsById()`? What SQL does it generate?
9. What is `deleteById()` vs `delete()`? What is `deleteAll()` vs `deleteAllInBatch()`?
10. What is `count()`?
11. What is `flush()`? When would you call it manually?
12. What is `@Repository`? Is it needed on a Spring Data JPA interface?
13. What is a custom repository fragment? How do you add custom methods to a repository?

Derived Query Methods & @Query

1. What is a derived query method? How does Spring Data parse the method name?
2. What keywords can you use: `findBy`, `getBy`, `readBy`, `countBy`, `existsBy`, `deleteBy`?
3. What are the condition keywords: `Between`, `LessThan`, `GreaterThan`, `Like`, `In`, `NotIn`, `IsNull`?
4. What are the combining keywords: `And`, `Or`, `Not`?
5. What is `OrderBy` in a derived query method?
6. What is `findTop3By...`? What is `findFirst5By...`?
7. What is `@Query`? How do you write a JPQL query?
8. What is a named parameter in `@Query` using `:paramName` and `@Param`?
9. What is a positional parameter in `@Query` using `?1` `?2`?
10. What is `nativeQuery=true` in `@Query`? When would you use it?
11. What is `@Modifying`? When is it required? What must always accompany it?
12. What is `clearAutomatically=true` in `@Modifying`? What does it do?
13. What is `@Query` with a DELETE statement? Is `@Modifying` needed?
14. What is `@NamedQuery`? How is it different from `@Query`?
15. What is the Criteria API? When would you use it over `@Query`?
16. What is Specifications (`JpaSpecificationExecutor`)? How do you build dynamic queries?
17. What is QueryDSL? How does it integrate with Spring Data?

Pagination, Sorting & Projections

1. What is `Pageable`? What information does it carry?
2. What is `PageRequest`? How do you create a `PageRequest` with sorting?
3. What is `Page`? What information does it contain?

4. What is Slice? How does it differ from Page?
5. What is Sort? How do you create a Sort by multiple fields?
6. What is Sort.Direction? What is Sort.Order?
7. How do you expose pagination as query parameters in a REST controller?
8. What is a projection in Spring Data JPA?
9. What is an interface-based projection? Write an example.
10. What is a class-based projection (DTO projection)? Write an example.
11. What is a dynamic projection? How do you use generics for it?
12. What is @Value in a projection interface? How is it used for computed fields?
13. What is @EntityGraph in a repository method?
14. What is a named entity graph with @NamedEntityGraph?

Spring Security Architecture

1. What is Spring Security? What does it protect against?
2. What is the Security Filter Chain? How does it wrap the servlet container?
3. What is DelegatingFilterProxy? What is FilterChainProxy?
4. What is the order of key security filters in the default chain?
5. What is SecurityContext? What is SecurityContextHolder?
6. What is ThreadLocal? How does SecurityContextHolder use it?
7. What is the Authentication interface? What does it contain?
8. What is AuthenticationManager? What is ProviderManager?
9. What is AuthenticationProvider?
10. What is DaoAuthenticationProvider?
11. What is UserDetails? What is UserDetailsService?
12. What is UsernamePasswordAuthenticationToken?
13. What is GrantedAuthority? What is the difference between a role and an authority?
14. What is the ROLE_ prefix convention?
15. What is SecurityContextPersistenceFilter?
16. What is ExceptionTranslationFilter?
17. What is AuthorizationFilter (Spring Security 5.5+)?
18. What is OncePerRequestFilter? Why should security filters extend it?
19. What is a SecurityFilterChain bean? How do you customise it?

JWT, BCrypt & Authorisation

1. What is JWT? What are its 3 parts: header, payload, signature?
2. What claims are in a JWT payload? What are registered claims (iss, sub, aud, exp, iat)?
3. What is the difference between HS256 and RS256 signing algorithms?
4. How do you validate a JWT: signature check, expiry, claim extraction?
5. What is stateless authentication? Why is JWT suitable for it?
6. What is a refresh token? How is it used to renew an access token?
7. What is the login flow: request → validate credentials → generate JWT → return token?
8. What is the JWT validation flow per request: extract → parse → set SecurityContext?
9. How do you extract the Bearer token from the Authorization header?
10. How do you store JWT on the client — localStorage vs httpOnly cookie? Pros and cons?
11. What is jjwt? What are the main classes: JwtBuilder, Claims, Jwts?
12. What is BCryptPasswordEncoder? What is the strength parameter?
13. What is PasswordEncoder interface? What other implementations exist?
14. What is CSRF? Why do browsers enforce it?
15. Why is CSRF disabled for stateless JWT-based REST APIs?

16. What is CORS in the context of Spring Security? How do you enable it?
17. What is SessionCreationPolicy.STATELESS? What does it prevent?
18. What is @PreAuthorize? Give 3 different expression examples.
19. What is @PostAuthorize? When would you use it?
20. What is @Secured? How is it different from @PreAuthorize?
21. What is @EnableMethodSecurity? What is it replacing?
22. What is hasRole() vs hasAuthority() in security expressions?
23. What is permit All() vs authenticated() vs anonymous()?
24. What is OAuth 2.0? What are its grant types?
25. What is OIDC? How does it extend OAuth 2.0?

AOP Fundamentals

1. What is AOP? What cross-cutting concerns does it handle?
2. What is an Aspect? What is Advice? What is a Pointcut? What is a JoinPoint?
3. What is Weaving? What are the 3 types (compile-time, load-time, runtime)?
4. What is the difference between Spring AOP and AspectJ?
5. What JoinPoints does Spring AOP support?
6. What are the limitations of Spring AOP compared to full AspectJ?
7. What is `@EnableAspectJAutoProxy`?
8. What is `proxyTargetClass=true` in `@EnableAspectJAutoProxy`?
9. What is a JDK dynamic proxy? When does Spring use it?
10. What is a CGLIB proxy? When does Spring use it?
11. What is the self-invocation problem in Spring AOP? How do you solve it?
12. What is `@Aspect`? What makes a bean an Aspect?

Pointcut Expressions & Advice

1. What is the `execution()` pointcut expression? Break down every part of the syntax.
2. What is `within()` pointcut expression?
3. What is `@annotation()` pointcut? Give a use case.
4. What is `@within()` pointcut? How is it different from `within()`?
5. What is `bean()` pointcut expression?
6. What is `args()` pointcut expression?
7. How do you combine pointcut expressions with `&&`, `||`, and `!?`
8. What is `@Pointcut`? How do you declare and reuse a named pointcut?
9. What is `@Before`? What can it do?
10. What is `@After`? When does it run? Is it like `finally`?
11. What is `@AfterReturning`? How do you access the return value using `returning` attribute?
12. What is `@AfterThrowing`? How do you access the exception using `throwing` attribute?
13. What is `@Around`? What is `ProceedingJoinPoint`? How do you call `proceed()`?
14. Can `@Around` suppress an exception? Can it change the return value?
15. What is the execution order when multiple `@Aspect` beans advise the same method?
16. What is `@Order`? How do you control the order of aspects?
17. What is `JoinPoint.getSignature()`? What is `JoinPoint.getArgs()`?
18. How do you pass arguments from the pointcut to the advice method using `args()`?
19. What is an introduction (inter-type declaration) in AspectJ?
20. What is `@DeclareParents`? What does it do?

JUnit 5 with Spring

1. What is `@ExtendWith(SpringExtension.class)`? What does it register?
2. What is `@SpringBootTest`? What does it load?
3. What is `webEnvironment=MOCK` vs `RANDOM_PORT` vs `DEFINED_PORT` vs `NONE`?
4. What is `TestRestTemplate`? When is it used over `MockMvc`?
5. What is `WebTestClient`? When is it preferred?
6. What is `@WebMvcTest`? What layers does it load?
7. What is `MockMvc`? What is `@AutoConfigureMockMvc`?
8. How do you perform a GET request with `MockMvc` and assert a 200 response?
9. How do you POST JSON with `MockMvc` and assert the response body?
10. What is `MockMvcRequestBuilders`? What is `MockMvcResultMatchers`?
11. What is `@DataJpaTest`? What does it configure? What database does it use?
12. How do you use `@DataJpaTest` with a real MySQL database (`replace=NONE`)?
13. What is `@TestConfiguration`? How is it different from `@Configuration`?
14. What is `@MockBean`? What context does it register in?
15. What is `@SpyBean`? How does it differ from `@SpyBean`?
16. What is `@Sql`? How do you pre-populate test data?
17. What is `@SqlGroup`?
18. What is `@DirtiesContext`? When is it needed?
19. What is `@ActiveProfiles`? How is it used in test classes?
20. What is `Testcontainers`? How do you use `@Testcontainers` and `@Container`?
21. What is `@SpringBootTest` with `@Testcontainers` for integration testing?

Mockito with Spring

1. What is `@Mock` vs `@MockBean`? When do you use each?
2. What is `@InjectMocks`? How does Mockito inject dependencies?
3. What is `@Spy` vs `@SpyBean`?
4. What is `@Captor` / `ArgumentCaptor`? Give an example of when it is needed.
5. What is `when().thenReturn()` vs `doReturn().when()`? What is the difference?
6. What is `when().thenThrow()` vs `doThrow().when()`?
7. What is `verify(mock, times(n))`? What is `verify(mock, never())`?
8. What is `inOrder()`? How do you verify method call sequence?
9. What is `ArgumentMatchers.any()`, `anyString()`, `eq()`, `isNull()`?
10. What is `Mockito.reset(mock)`? Is it a best practice?
11. What is `mockStatic()`? How do you mock static methods in Mockito 3.4+?
12. What is `@ExtendWith(MockitoExtension.class)`?
13. What is `BDDMockito`? How is `given().willReturn()` different from `when().thenReturn()`?

Transaction Management

1. What is the difference between programmatic and declarative transaction management?
2. What is `@Transactional`? How does Spring implement it using AOP proxy?
3. What is the default rollback rule for `@Transactional`?
4. How do you rollback for a checked exception in `@Transactional`?
5. How do you prevent rollback for a specific `RuntimeException`?
6. What is `readOnly=true` in `@Transactional`? What optimisations does Hibernate make?
7. What is the self-invocation problem with `@Transactional`? Why does it fail?
8. What is `@Transactional` on a private or final method — does it work?
9. What is `PlatformTransactionManager`? Name 3 implementations.
10. What is `TransactionTemplate` for programmatic transaction management?
11. What is `@TransactionalEventListener`? How is it different from `@EventListener`?

Transaction Propagation

1. What is transaction propagation? Why does it matter?
2. What is `REQUIRED` propagation? What happens if a transaction already exists?
3. What is `REQUIRES_NEW` propagation? What happens to the outer transaction?
4. What is `NESTED` propagation? What is a savepoint?
5. What is `MANDATORY` propagation? What exception does it throw?
6. What is `SUPPORTS` propagation? What is `NOT_SUPPORTED`?
7. What is `NEVER` propagation?
8. Give a real-world scenario where `REQUIRES_NEW` is needed for audit logging.
9. What happens if an inner `@Transactional(propagation=REQUIRED)` throws an exception?
10. What is a rollback-only transaction? What is `UnexpectedRollbackException`?

Transaction Isolation

1. What are the 4 transaction isolation levels in Spring (`@Transactional`)?
2. What is `READ_UNCOMMITTED`? What is a dirty read?
3. What is `READ_COMMITTED`? What anomaly does it prevent?
4. What is `REPEATABLE_READ`? What anomaly does it prevent?
5. What is `SERIALIZABLE`? What does it prevent? What is the performance cost?
6. What is the difference between a dirty read, non-repeatable read, and phantom read?
7. What is the default isolation level in Spring? What is MySQL's default?
8. What is optimistic vs pessimistic locking? How do you enable each in Spring?
9. What is `@Lock` in Spring Data JPA? What are the `LockModeType` values?

Spring REST Advanced

1. What is `@Async`? How do you enable it in Spring Boot?
2. What is `@EnableAsync`? What is the default thread pool for `@Async`?
3. What is `CompletableFuture` as a return type in `@Async` methods?
4. What is `DeferredResult` in Spring MVC? How does it enable async processing?
5. What is `Callable` as a return type in a Spring MVC controller?
6. What is Server-Sent Events (SSE)? What is `SseEmitter`?
7. What is `ResponseBodyEmitter`? What is `StreamingResponseBody`?
8. What is `@Scheduled`? What is the difference between `fixedDelay`, `fixedRate`, and `cron`?
9. What is `@EnableScheduling`?
10. What is a cron expression format in Spring?
11. What is the Spring Cache abstraction? What is `@EnableCaching`?
12. What is `@Cacheable`? How does Spring decide whether to execute the method or return cached result?
13. What is `@CachePut`? How is it different from `@Cacheable`?
14. What is `@CacheEvict`? What is `allEntries=true`?
15. What is `@Caching`? When do you use it?
16. What cache providers does Spring support?

Spring WebFlux & Reactive

1. What is Spring WebFlux? How does it differ from Spring MVC?
2. What is Project Reactor? What is `Mono` and `Flux`?
3. What is the event loop model in WebFlux?
4. What is backpressure in reactive streams?
5. What is `WebClient`? How does it differ from `RestTemplate`?
6. What is `R2DBC`? Why is it needed for reactive database access?
7. What is the `@GetMapping` returning `Flux` in WebFlux?
8. What is `RouterFunction`? What is `HandlerFunction`?
9. When should you choose WebFlux over Spring MVC?
10. What is `WebSocket` in Spring? What is `@MessageMapping`?
11. What is STOMP protocol? What is `@EnableWebSocketMessageBroker`?

Bean Definition & Component Scanning Annotations

Annotation	Purpose	Package / Layer
@Component	Generic stereotype — registers class as Spring bean	<i>org.springframework.stereotype</i>
@Service	Business logic bean (extends @Component, semantic)	<i>org.springframework.stereotype</i>
@Repository	DAO bean + JPA exception translation	<i>org.springframework.stereotype</i>
@Controller	Spring MVC web controller (returns views)	<i>org.springframework.stereotype</i>
@RestController	@Controller + @ResponseBody (returns JSON/XML)	<i>org.springframework.web.bind.annotation</i>
@Configuration	Marks class as source of @Bean definitions	<i>org.springframework.context.annotation</i>
@Bean	Registers method return value as a Spring bean	<i>org.springframework.context.annotation</i>
@ComponentScan	Enables component scanning for @Component classes	<i>org.springframework.context.annotation</i>
@Import	Imports one or more @Configuration classes	<i>org.springframework.context.annotation</i>
@ImportResource	Imports Spring XML config files	<i>org.springframework.context.annotation</i>
@PropertySource	Loads properties from a .properties file	<i>org.springframework.context.annotation</i>
@PropertySources	Repeatable @PropertySource container	<i>org.springframework.context.annotation</i>

Dependency Injection Annotations

Annotation	Purpose	Notes
@Autowired	Inject bean by type (field, setter, constructor)	<i>Can be optional with required=false</i>
@Qualifier('name')	Specify bean name when multiple candidates exist	<i>Used with @Autowired</i>
@Primary	Mark default bean when multiple of same type exist	<i>Overridden by @Qualifier</i>
@Value('\${key}')	Inject property value or SpEL expression	<i>Use \${} for props, #{} for SpEL</i>
@Lazy	Delay bean initialisation until first use	<i>Also helps with circular dependency</i>
@Required	Mark setter as required (deprecated Spring 5)	<i>Legacy — use constructor injection</i>
@Inject	JSR-330 equivalent of @Autowired	<i>javax.inject package</i>
@Named	JSR-330 equivalent of @Component / @Qualifier	<i>javax.inject package</i>
@Resource	JSR-250 — inject by name first, then by type	<i>javax.annotation package</i>
@ConfigurationProperties	Bind properties to POJO fields (relaxed binding)	<i>Use with @EnableConfigurationProperties</i>

<code>@EnableConfigurationProperties</code>	Enable <code>@ConfigurationProperties</code> beans	Usually on <code>@SpringBootApplication</code>
<code>@ConstructorBinding</code>	Force constructor-based binding in <code>@ConfigurationProperties</code>	Spring Boot 2 — auto in Boot 3

Bean Lifecycle & Scope Annotations

Annotation	Purpose	Notes
<code>@PostConstruct</code>	Run method after bean fully initialised	JSR-250 — <i>jakarta.annotation</i>
<code>@PreDestroy</code>	Run method before bean destroyed	JSR-250 — <i>jakarta.annotation</i>
<code>@Scope('singleton')</code>	One shared instance per <code>ApplicationContext</code>	Default scope
<code>@Scope('prototype')</code>	New instance on every <code>getBean()</code> / <code>@Autowired</code>	No destroy lifecycle
<code>@Scope('request')</code>	One instance per HTTP request (web only)	Needs scoped proxy for singleton
<code>@Scope('session')</code>	One instance per HTTP session	Needs scoped proxy for singleton
<code>@Scope('application')</code>	One instance per <code>ServletContext</code>	Rarely used
<code>@DependsOn('beanName')</code>	Force bean to initialise after named bean	Explicit ordering
<code>@Lazy</code>	Create bean on first access, not at startup	Also on <code>@Autowired</code> injection point
<code>@Profile('dev')</code>	Register bean only when named profile is active	Can negate: <code>@Profile("!prod")</code>

Conditional Annotations

Annotation	Condition for bean registration	Usage context
<code>@Conditional(Condition.class)</code>	Programmatic condition via <code>Condition</code> interface	Base of all <code>@ConditionalOn*</code>
<code>@ConditionalOnClass</code>	Activates if specified class is on classpath	Auto-configuration
<code>@ConditionalOnMissingClass</code>	Activates if specified class is NOT on classpath	Auto-configuration
<code>@ConditionalOnBean</code>	Activates if specified bean exists in context	Auto-configuration ordering
<code>@ConditionalOnMissingBean</code>	Activates if specified bean does NOT exist	Allows user override
<code>@ConditionalOnProperty</code>	Activates based on property value	feature toggles
<code>@ConditionalOnExpression</code>	Activates based on SpEL expression	Complex conditions
<code>@ConditionalOnWebApplication</code>	Activates in web context	MVC-specific auto-config
<code>@ConditionalOnNotWebApplication</code>	Activates in non-web context	Batch / CLI apps
<code>@ConditionalOnJava</code>	Activates based on Java version	JVM version conditions
<code>@ConditionalOnResource</code>	Activates if resource (file) is present on classpath	Config file detection
<code>@ConditionalOnSingleCandidate</code>	Activates if exactly one bean of type exists	Primary bean selection

Spring Boot Core Annotations

Annotation	Purpose	Notes
@SpringBootApplication	Combines @Configuration + @EnableAutoConfiguration + @ComponentScan	Entry point class
@EnableAutoConfiguration	Triggers Spring Boot auto-configuration mechanism	Part of @SpringBootApplication
@SpringBootConfiguration	Specialised @Configuration for Spring Boot	Part of @SpringBootApplication
@AutoConfiguration	Marks auto-configuration class (Spring Boot 3+)	Registered in AutoConfiguration.imports
@AutoConfigureBefore	Auto-config runs before named class	Ordering in auto-config
@AutoConfigureAfter	Auto-config runs after named class	Ordering in auto-config
@AutoConfigureOrder	Explicit numeric order for auto-configuration	Fine-grained ordering
@SpringBootTest	Loads full ApplicationContext for integration test	webEnvironment param controls server
@WebMvcTest	Loads only web layer (@Controller, security)	Use @MockBean for services
@DataJpaTest	Loads JPA layer only, H2 by default	Use replace=NONE for real DB
@WebFluxTest	Loads WebFlux layer only	For reactive controller tests
@RestClientTest	Tests REST clients (@RestTemplate)	Mocks server
@JsonTest	Tests JSON serialization/deserialization	Uses JacksonTester
@DataMongoTest	Loads MongoDB layer only	Similar to @DataJpaTest
@DataRedisTest	Loads Redis layer only	For Redis repository tests
@MockBean	Creates Mockito mock + registers in Spring context	Replaces real bean in context
@SpyBean	Creates Mockito spy + registers in Spring context	Wraps real bean

Spring Boot Feature Annotations

Annotation	Purpose	Notes
@EnableScheduling	Enables @Scheduled task execution	Add to @Configuration class
@Scheduled(fixedRate=)	Run method at fixed interval (ms)	Measured from start of last invocation
@Scheduled(fixedDelay=)	Run method with fixed delay after completion	Measured from end of last invocation
@Scheduled(cron=)	Run on cron schedule	6-field cron: sec min hr day month weekday
@EnableAsync	Enables async method execution	Add to @Configuration
@Async	Run method in separate thread	Returns void, Future, or CompletableFuture
@EnableCaching	Enables Spring Cache abstraction	Add to @Configuration
@Cacheable	Cache method return value	key, condition, unless attributes
@CachePut	Always execute and update cache	Used for update operations

@CacheEvict	Remove entry/entries from cache	<i>allEntries=true clears whole cache</i>
@Caching	Group multiple caching annotations	<i>For complex scenarios</i>
@CacheConfig	Class-level cache defaults	<i>Sets cacheNames, keyGenerator</i>
@EnableTransactionManagement	Enables @Transactional annotation processing	<i>Usually auto-enabled in Boot</i>
@Transactional	Wrap method in DB transaction	<i>See S11 for all attributes</i>
@EnableJpaRepositories	Enable JPA repository scanning	<i>Usually auto-configured</i>
@EnableMongoRepositories	Enable MongoDB repository scanning	<i>Spring Data MongoDB</i>
@ActiveProfiles	Set active profiles for a test class	<i>@ActiveProfiles("test")</i>
@TestPropertySource	Override properties for a test	<i>locations or properties attribute</i>

Request Mapping Annotations

Annotation	Purpose	HTTP Method
@RequestMapping	Base URL mapping — class or method level	Any (use method attribute)
@GetMapping	Map HTTP GET request	GET
@PostMapping	Map HTTP POST request	POST
@PutMapping	Map HTTP PUT request (full update)	PUT
@PatchMapping	Map HTTP PATCH request (partial update)	PATCH
@DeleteMapping	Map HTTP DELETE request	DELETE
@PathVariable	Extract value from URI path segment {id}	Path: /users/{id}
@RequestParam	Extract query string parameter ?key=value	Query: ?status=ACTIVE
@RequestBody	Deserialise JSON request body to object	Requires Content-Type: application/json
@ResponseBody	Serialise return value to JSON response body	Part of @RestController
@RequestHeader	Extract value from HTTP request header	Authorization, Content-Type, etc.
@CookieValue	Extract value from HTTP cookie	JWT in cookie pattern
@MatrixVariable	Extract matrix variable from path segment	Rare use case
@ModelAttribute	Bind request params to a model object (form data)	Used in form-based MVC
@SessionAttribute	Access session attribute from SecurityContext / session	Read existing session attrs
@SessionAttributes	Define which model attributes are stored in session	Class-level annotation
@ResponseStatus	Set default HTTP status for a method or exception class	@ResponseStatus(HttpStatus.CREATED)
@CrossOrigin	Enable CORS for a controller or method	origins, methods, maxAge
@RequestPart	Bind multipart request part	File upload with metadata
@InitBinder	Customise WebDataBinder for a controller	Custom editors / formatters
@ExceptionHandler	Handle specific exception in a controller or @ControllerAdvice	Scoped or global
@ControllerAdvice	Global exception handler + model attributes for all controllers	Class-level
@RestControllerAdvice	@ControllerAdvice + @ResponseBody	For REST APIs

Bean Validation Annotations

Annotation	Validates	Notes
@Valid	Triggers Bean Validation on method parameter	Cascade to nested objects
@Validated	Spring variant — supports validation groups	On class or method parameter
@NotNull	Field must not be null	Does not check empty string
@NotEmpty	String/Collection must not be null or empty	Length >= 1
@NotBlank	String must not be null, empty, or whitespace only	Trimmed length >= 1

@Size(min, max)	String/Collection size must be in range	<i>min default 0, max default MAX_INT</i>
@Min(value)	Number must be >= value	<i>For int, long, BigDecimal</i>
@Max(value)	Number must be <= value	<i>For int, long, BigDecimal</i>
@Positive	Number must be > 0	<i>Excludes 0</i>
@PositiveOrZero	Number must be >= 0	<i>Includes 0</i>
@Negative	Number must be < 0	
@NegativeOrZero	Number must be <= 0	
@Email	String must be valid email format	<i>Uses RFC 5322</i>
@Pattern(regex)	String must match regular expression	<i>Java regex syntax</i>
@Past	Date must be in the past	<i>Works with LocalDate, LocalDateTime</i>
@Future	Date must be in the future	<i>Works with LocalDate, LocalDateTime</i>
@PastOrPresent	Date must be past or present	
@FutureOrPresent	Date must be future or present	
@Digits(integer, fraction)	Number must have limited digits	<i>For DECIMAL fields</i>
@DecimalMin	Number must be >= decimal value	<i>For strings representing numbers</i>
@DecimalMax	Number must be <= decimal value	
@AssertTrue	Boolean must be true	<i>For complex custom logic</i>
@AssertFalse	Boolean must be false	
@Constraint	Meta-annotation for creating custom constraints	<i>With validatedBy attribute</i>

JPA / Hibernate Entity Annotations

Annotation	Purpose	Notes
@Entity	Marks Java class as JPA entity (mapped to DB table)	Requires no-arg constructor
@Table(name=)	Specify table name, schema, uniqueConstraints	Optional — defaults to class name
@Id	Primary key field	Required on every entity
@GeneratedValue	Auto-generate PK value	strategy: IDENTITY, SEQUENCE, TABLE, AUTO
@Column	Map field to DB column (name, nullable, unique, length)	Optional — defaults to field name
@Transient	Exclude field from persistence	Not mapped to any column
@Enumerated(String)	Map enum to VARCHAR in DB	Always use STRING not ORDINAL
@Lob	Map to CLOB (TEXT) or BLOB column	For large text or binary data
@Temporal	Map Date to DATE, TIME, or TIMESTAMP (legacy)	Use LocalDate/LocalDateTime instead
@CreationTimestamp	Auto-set to now on INSERT (Hibernate)	org.hibernate.annotations
@UpdateTimestamp	Auto-update to now on UPDATE (Hibernate)	org.hibernate.annotations
@Version	Optimistic locking — increment on each update	Long or Integer field
@Embedded	Embed an @Embeddable object's columns into this table	Value object pattern
@Embeddable	Mark class as embeddable into an @Entity	No separate table
@AttributeOverride	Override column mapping of @Embedded field	When same @Embeddable used twice
@IdClass	Composite PK using a separate ID class	Alternative to @EmbeddedId
@EmbeddedId	Composite PK as embedded object	PK class must be @Embeddable
@MapsId	Map @OneToOne with shared PK	Shared primary key pattern
@NaturalId	Business key (Hibernate)	Enables findByNaturalId
@ColumnDefault	DB default value for column (Hibernate DDL)	org.hibernate.annotations

JPA Relationship & Repository Annotations

Annotation	Purpose	Default Fetch Type
@ManyToOne	Many entities → one entity (FK side)	EAGER
@OneToMany(mappedBy=)	One entity → many entities (inverse side)	LAZY
@OneToOne	One-to-one entity relationship	EAGER
@ManyToMany	Many-to-many using join table	LAZY
@JoinColumn(name=)	Specify FK column name for @ManyToOne / @OneToOne	FK in current table
@JoinTable	Specify join table for @ManyToMany	name, joinColumns, inverseJoinColumns

@OrderBy	Order @OneToMany collection by JPQL expression	Applied on load
@OrderColumn	Persist List order in a DB column	Order preserved in DB
@Fetch(FetchMode)	Hibernate fetch mode: JOIN, SELECT, SUBSELECT	org.hibernate.annotations
@BatchSize(size=)	Load related entities in batches of N	Reduces N+1 queries
@EntityGraph	Specify fetch plan for a repository query	Overrides default fetch types
@NamedEntityGraph	Define named entity graph on entity class	Used with @EntityGraph
@Query	Custom JPQL or native SQL on repository method	Spring Data
@Modifying	Required for @Query with UPDATE or DELETE	clearAutomatically=true optional
@Param	Name a query parameter in @Query method	Matches :paramName in JPQL
@Lock	Set LockModeType on a repository query	PESSIMISTIC_WRITE, OPTIMISTIC, etc.
@Repository	Marks DAO interface or class (exception translation)	Not required on JpaRepository subinterfaces

Spring Security Annotations

Annotation	Purpose	Notes
@EnableWebSecurity	Enable Spring Security's web security support	On @Configuration class
@EnableMethodSecurity	Enable @PreAuthorize, @PostAuthorize, @Secured	Replaces @EnableGlobalMethodSecurity
@EnableGlobalMethodSecurity	Legacy — enable method-level security (Boot 2)	Deprecated in Boot 3
@PreAuthorize	Check expression BEFORE method executes	hasRole('ADMIN'), #id == authentication.principal.id
@PostAuthorize	Check expression AFTER method returns	Access returnObject
@Secured	Restrict method to specific roles (simple)	{'ROLE_ADMIN', 'ROLE_USER'}
@RolesAllowed	JSR-250 equivalent of @Secured	javax.annotation.security
@PermitAll	Allow all access (JSR-250)	No authentication required
@DenyAll	Deny all access (JSR-250)	Blocks all
@AuthenticationPrincipal	Inject current authenticated user / UserDetails	In controller method params
@CurrentSecurityContext	Inject SecurityContext or its fields	In controller method params
@WithMockUser	Test with a mock authenticated user	Spring Security Test
@WithUserDetails	Test with user loaded from UserDetailsService	Spring Security Test
@WithAnonymousUser	Test as anonymous user	Spring Security Test

Spring AOP & Transaction Annotations

Annotation	Purpose	Notes
@EnableAspectJAutoProxy	Enable AspectJ-style AOP proxy creation	proxyTargetClass=true forces CGLIB
@Aspect	Mark class as an AOP Aspect	Must also be a Spring bean (@Component)
@Pointcut	Declare a reusable named pointcut expression	Method must be void with no body
@Before	Advice runs before the matched method	Cannot prevent execution

@After	Advice runs after method (finally — regardless of outcome)	
@AfterReturning	Advice runs after successful return	<i>Access return value via returning attr</i>
@AfterThrowing	Advice runs when exception is thrown	<i>Access exception via throwing attr</i>
@Around	Advice wraps method — call proceed() to execute	<i>Most powerful advice type</i>
@EnableTransactionManagement	Enable @Transactional annotation processing	<i>proxyTargetClass attribute</i>
@Transactional	Wrap method in a database transaction	<i>propagation, isolation, readOnly, rollbackFor</i>
@TransactionalEventListener	Listen to event only within a transaction phase	<i>After commit, rollback, etc.</i>
@Order	Control execution order of multiple aspects or beans	<i>Lower value = higher priority</i>
@DeclareParents	Add interface implementation to target class (introduction)	<i>AspectJ feature</i>

Top 50 Annotations Every Spring Developer Must Know

Annotation	What it does in one sentence
@SpringBootApplication	Entry point — combines @Configuration + @EnableAutoConfiguration + @ComponentScan
@Component / @Service / @Repository / @Controller	Register class as a Spring bean (stereotype annotations)
@Autowired	Inject a dependency by type
@Qualifier('name')	Resolve ambiguity when multiple beans of same type exist
@Value('\${key}')	Inject a property value from application.properties
@ConfigurationProperties	Bind a block of properties to a POJO
@Bean	Register method return value as a Spring bean in a @Configuration class
@Configuration	Mark class as a source of bean definitions
@ComponentScan	Scan package for @Component annotated classes
@Conditional / @ConditionalOnClass	Register bean only when a condition is met
@Profile	Register bean only when a specific profile is active
@Scope('prototype')	Create a new bean instance every time it is requested
@PostConstruct / @PreDestroy	Run method after bean init / before bean destruction
@Transactional	Wrap method in a DB transaction (rollback on RuntimeException)
@RequestMapping / @GetMapping / @PostMapping	Map HTTP requests to controller methods
@PathVariable / @RequestParam / @RequestBody	Bind HTTP path, query, or body to method parameters
@ResponseBody	Write method return value directly to HTTP response (JSON)
@ResponseStatus	Set default HTTP status code for a method or exception class
@RestController	@Controller + @ResponseBody for REST APIs
@RequestHeader	Bind HTTP header value to method parameter
@ExceptionHandler	Handle specific exception in controller or @ControllerAdvice
@RestControllerAdvice	Global exception handler that returns JSON
@CrossOrigin	Enable CORS for controller or method
@Valid / @Validated	Trigger Bean Validation on @RequestBody or method param
@NotNull / @NotBlank / @Size / @Email	Bean Validation constraints on DTO fields
@Entity / @Table	Map Java class to a database table
@Id / @GeneratedValue	Define primary key and its generation strategy
@Column	Map field to a DB column with constraints
@ManyToOne / @OneToMany / @ManyToMany	Define JPA entity relationships

@JoinColumn	Specify FK column name for a relationship
@Enumerated(EnumType.STRING)	Store enum as string in DB (not ordinal)
@Transient	Exclude field from JPA persistence
@CreationTimestamp / @UpdateTimestamp	Auto-set timestamps on insert/update (Hibernate)
@Version	Enable optimistic locking on an entity
@Query	Custom JPQL or native SQL query on a repository method
@Modifying	Required with @Query for UPDATE/DELETE operations
@Param	Name a parameter for @Query binding
@EnableWebSecurity	Enable Spring Security web configuration
@PreAuthorize	Check Spring Security expression before method execution
@Secured	Restrict method to specific roles
@AuthenticationPrincipal	Inject current authenticated user into controller method
@EnableMethodSecurity	Enable method-level security annotations
@Aspect / @Around / @Before / @AfterReturning	Define AOP Aspect and advice types
@Pointcut	Declare reusable named pointcut expression
@EnableAspectJAutoProxy	Enable Spring AOP via AspectJ annotations
@Scheduled	Run method on a schedule (cron, fixedRate, fixedDelay)
@EnableScheduling	Enable @Scheduled task execution
@Async	Run method asynchronously in a separate thread
@EnableAsync	Enable @Async processing
@Cacheable / @CacheEvict / @CachePut	Cache method result / evict / always update cache

INTERVIEW TIP: For every annotation, know 3 things: (1) What it does, (2) Which package/layer it belongs to, (3) A real example from your EnergyGrid project. The most commonly asked annotation questions: What is @Transactional propagation? What is @ConditionalOnMissingBean? What is the difference between @Mock and @MockBean? What is @Autowired by type vs @Qualifier by name? What is @PreAuthorize vs @Secured?